

Agentic Coding

Sven Helmer

Kathrin Paschen

Chapter 1

Introduction

Motivation

- There's a lot of hype around vibe coding at the moment

“No more humans coding in 5 years.”

Emad Mostaque

“AI will write most of the code soon and perform better than top coders.”

Mark Zuckerberg

“It's our job to create computing technology such that nobody has to program.”

Jensen Huang

Motivation (2)

- We have a slightly different opinion
- This is not the first hype we have seen



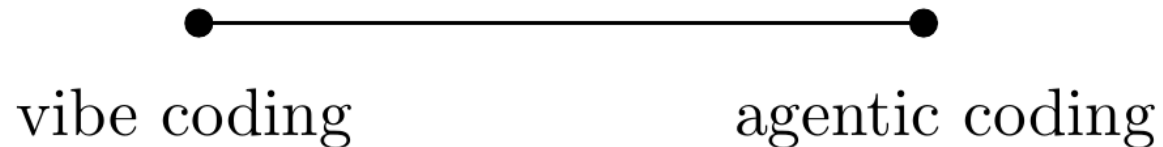
- We will try to bring some substance to this discussion

Things are Definitely Changing

- In the foreseeable future, AI will not replace developers
 - A developer using AI will replace one who doesn't
 - and developers (already) work differently
- Currently, AI boosts the skills of experienced experts
- It will not save you if you have no clue what you're doing

Basic Terminology

- Coding with AI covers a whole spectrum
 - Vibe coding: prompt and hope for the best
 - Agentic coding: a systematic approach that still utilizes software development principles



Vibe Coding

- Vibe coding typically looks like the following:
 - Write a prompt, e.g. “Build an app that does X.”
 - LLM generates some code
 - User tries it out, doesn't quite work
 - User writes another prompt requesting some changes
 - LLM changes code
 - Wash, rinse, repeat

Agentic Coding

- This is not about writing simple prompts
- Gives an LLM specific software development tasks in a systematic way, such as:
 - Formulate goal, e.g., in form of a specification
 - Devise a plan, break up general task into subtasks
 - Generate code
 - Validate code, e.g., by running tests
 - If necessary, revise implementation (or even planning)

Vibe-Coding vs. Agentic Coding

- That doesn't necessarily mean that vibe coding is bad and agentic coding is good
- Vibe coding:
 - makes sense for rapid prototyping
 - is for quickly trying out things
- Agentic coding
 - is a thorough process
 - creates reliable code

What Can Go Wrong?

- AI gets stuck in infinite loops
- Deliver incorrect solutions confidently
- Cheat: optimize for validation such as benchmarks
- Implements what you said, not what you meant
 - General problem, existed before AI

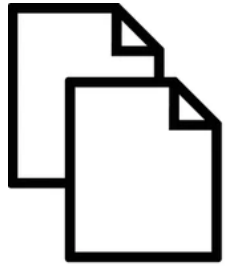
Summary

- Then: translate ideas into code (programming)
- Now: translate ideas into prompts (vibe-coding)
- Then: (systematically) applying principles and methods to create reliable software (software engineering)
- Now: give an AI (software-engineering) goals and constraints to create software (agentic coding)
- Important question: how much of the software-engineering process can be delegated?

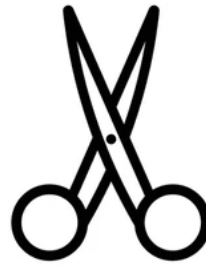
Chapter 2

The Environment

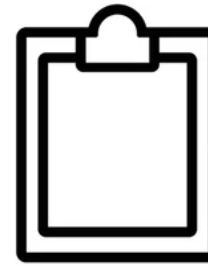
Basics



Copy



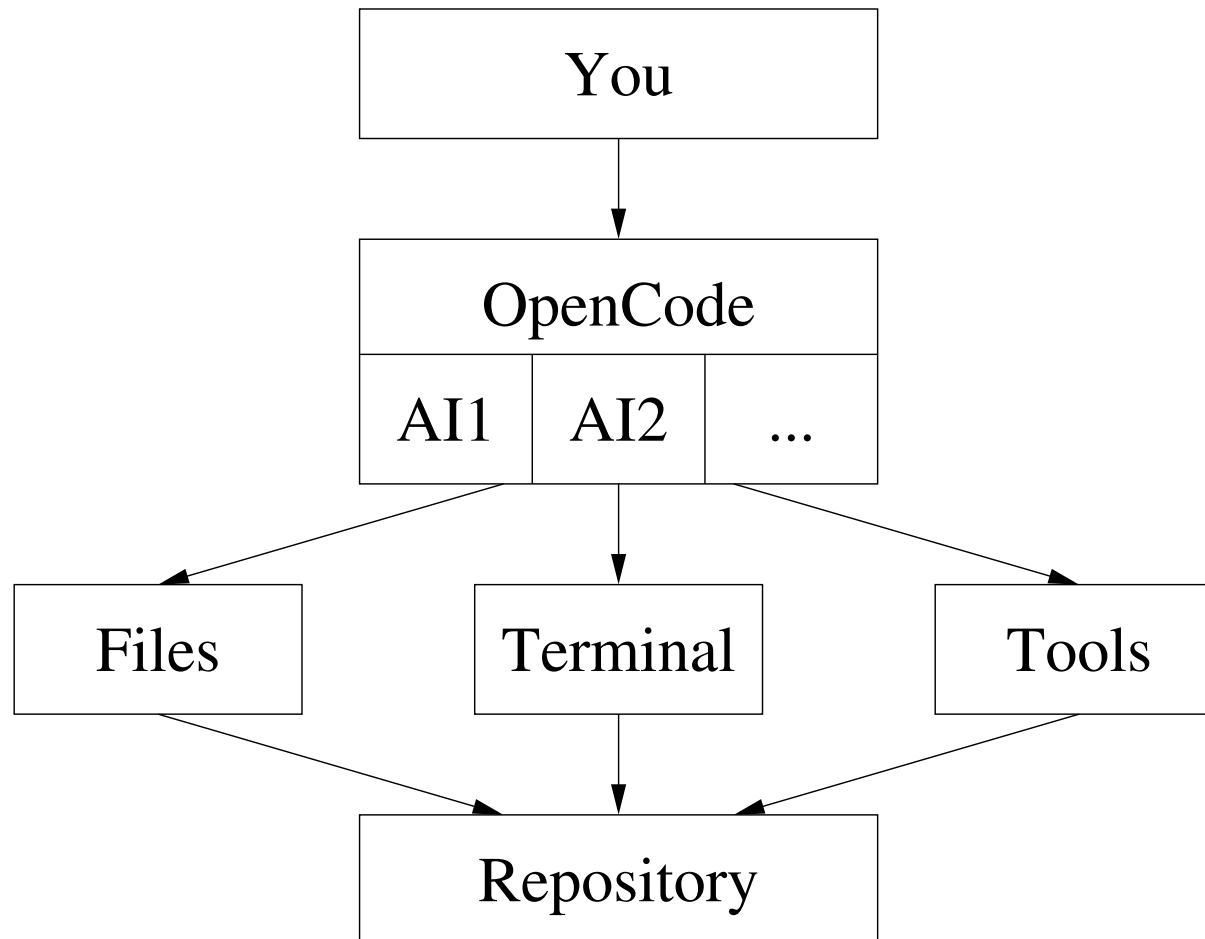
Cut



Paste

- What we don't want to do:
 - Prompting LLM on a web interface
 - Then copy/pasting code into file
- We use a more sophisticated environment

Overview



OpenCode

- OpenCode is an open-source coding tool
- Connects LLMs to development tools
- Model-agnostic: can run (almost) every model you want
- By default, stores code locally

The logo for OpenCode, featuring the word "open" in a light gray, lowercase, sans-serif font, followed by the word "code" in a bold, black, lowercase, sans-serif font. The letters are closely spaced and have a modern, blocky appearance.

Interaction with Files

- OpenCode is not an editor:
 - You can use any editor you want (VS Code, Emacs, IntelliJ) to look at the files
 - OpenCode executes commands in a terminal
- With these commands it can manipulate files and use other tools
 - This has direct implications on security

Repositories

- A repository is where a version control system (VCS) stores its data
- A VCS tracks and manages changes to files
 - Heavily used in managing source code
- Allows to revert to a previous state
- Facilitates collaboration

Summary

- This chapter is kept short on purpose
- We show how to set up and use this environment in hands-on exercises



THEORY TELLS. PRACTICE TEACHES.





